

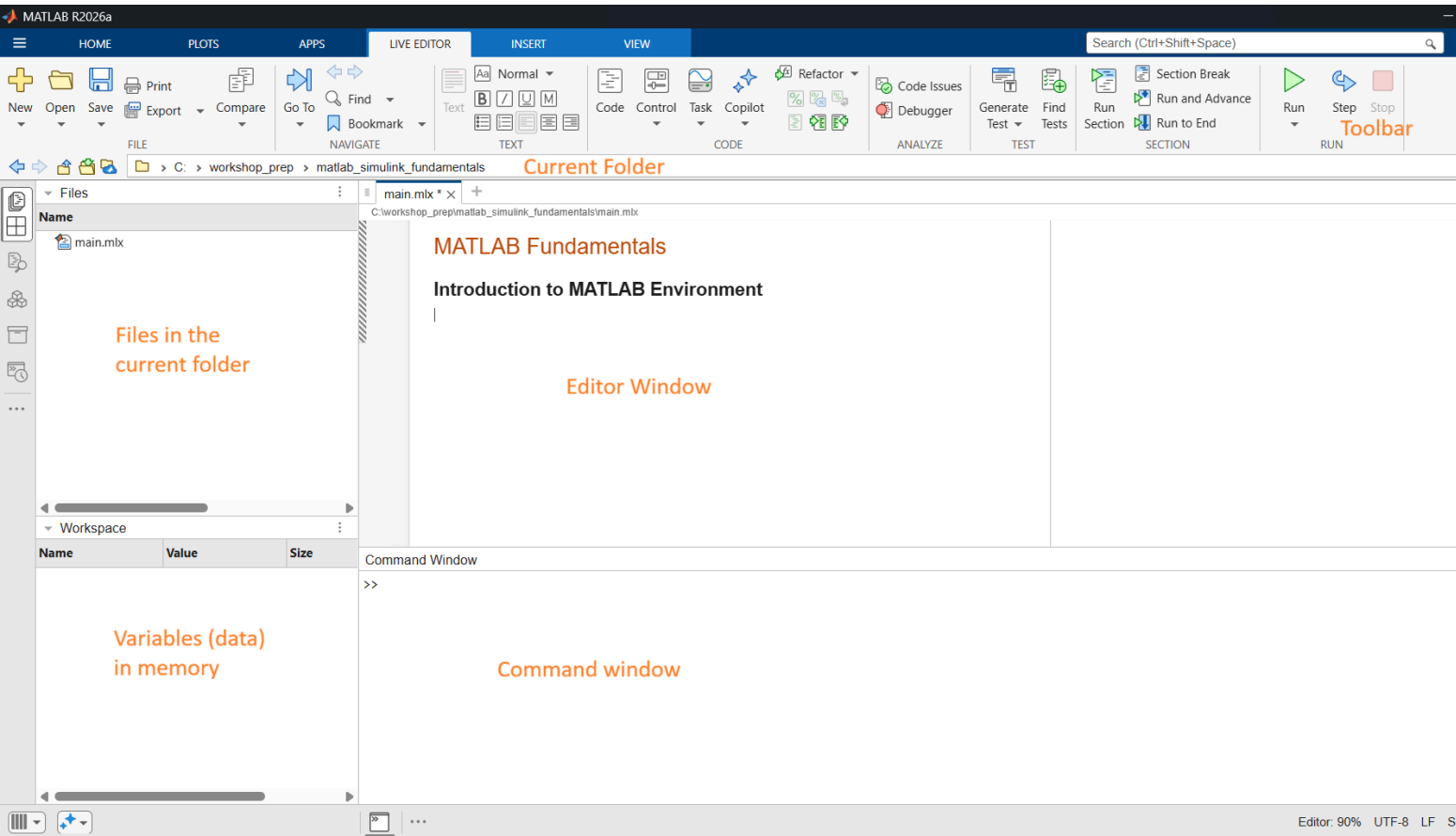
MATLAB Fundamentals

Table of Contents

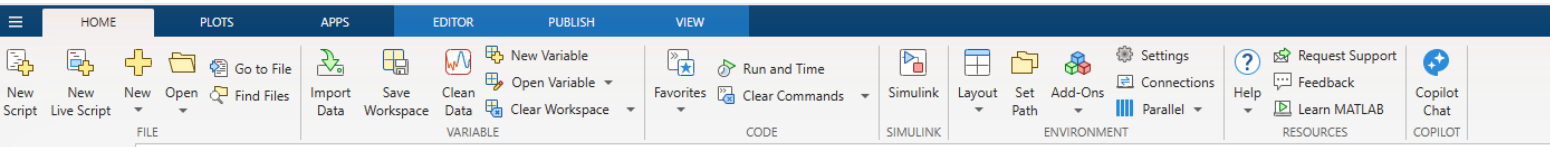
1. Introduction to MATLAB Environment.....	1
2. Basic MATLAB Commands.....	2
3. Working with Live Scripts.....	3
4. MATLAB Functions.....	3
5. Introduction to Project.....	4

1. Introduction to MATLAB Environment

The MATLAB desktop is organized into a few key areas: the Current Folder browser (files in your working directory), the Editor (write scripts, functions and live scripts), the Command Window (type commands at the >> prompt for immediate execution), and the Workspace (variables currently in memory).



The Toolstrip across the top groups common actions. On the HOME tab you can create scripts and live scripts, import data, manage the workspace, change the desktop Layout, open Settings, install Add-Ons, get Help, and launch Simulink.



2. Basic MATLAB Commands

Commands typed in the Command Window run immediately. Try these:

```
a = 10           % create a scalar variable
b = 2.5;         % semicolon suppresses the output
c = a * b        % arithmetic with variables
M = magic(3)     % a 3x3 matrix
s = "Hello Workshop" % a string
```

```
a = 10           % create a scalar variable
```

```
a =
10
```

```
b = 2.5;         % semicolon suppresses the output
```

Ending a line with a semicolon suppresses the printed output; without it, MATLAB echoes the result. Your Command Window should look like this:



```
Command Window
a =
10
>> b = 2.5;
>> c = a * b
c =
25
>> M = magic(3)
M =
     8     1     6
     3     5     7
     4     9     2
>> s = "Hello Workshop"
s =
"Hello Workshop"
>> |
```

Every variable you create is stored in the Workspace. Open the Workspace panel (type workspace, or use the Layout menu) to inspect names, values, sizes and classes:

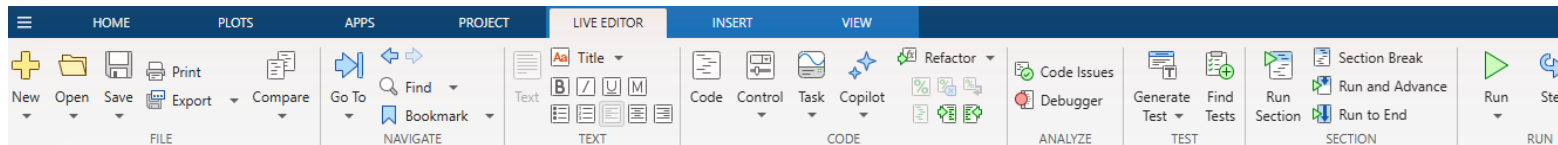
Workspace				
Name	Value	Size	Class	
 a	10	1×1	double	
 b	2.5000	1×1	double	
 c	25	1×1	double	
 M	[8,1,6;3,5,7;4,9,2]	3×3	double	
 s	"Hello Workshop"	1×1	string	

Useful housekeeping commands:

1. *whos* lists variables with details
2. *clc* clears the Command Window
3. *clear* removes all variables
4. *doc* followed by a name opens the documentation

3. Working with Live Scripts

A live script (.mlx) combines formatted text, images, equations, runnable code and its output in one document - this very file is a live script. When a live script is open, the LIVE EDITOR tab appears in the toolstrip:



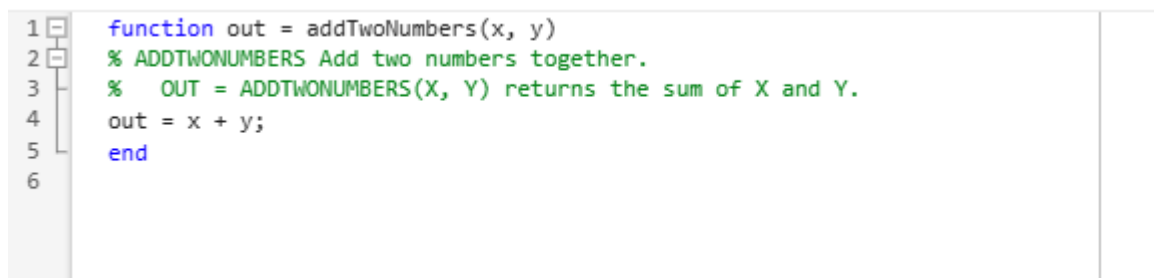
Things to try: insert text and code from the TEXT and CODE groups; run everything with Run or one section at a time with Run Section; structure your demo with Section Break; and share your work via Export (PDF, Word, HTML).

4. MATLAB Functions

Functions package reusable logic with defined inputs and outputs. A function file starts with the function keyword and is saved under the same name as the function:

```
function out = addTwoNumbers(x, y)
% ADDTWO NUMBERS Add two numbers together.
% OUT = ADDTWO NUMBERS(X, Y) returns the sum of X and Y.
out = x + y;
end
```

Save it as addTwoNumbers.m - in this project it lives in the Utilities folder. The Editor shows it with syntax highlighting:



Because Utilities is on the project path, the function can be called from anywhere - the Command Window, scripts, or other functions:

```
>> total = addTwoNumbers(10, 32)

total =

    42

>>
```

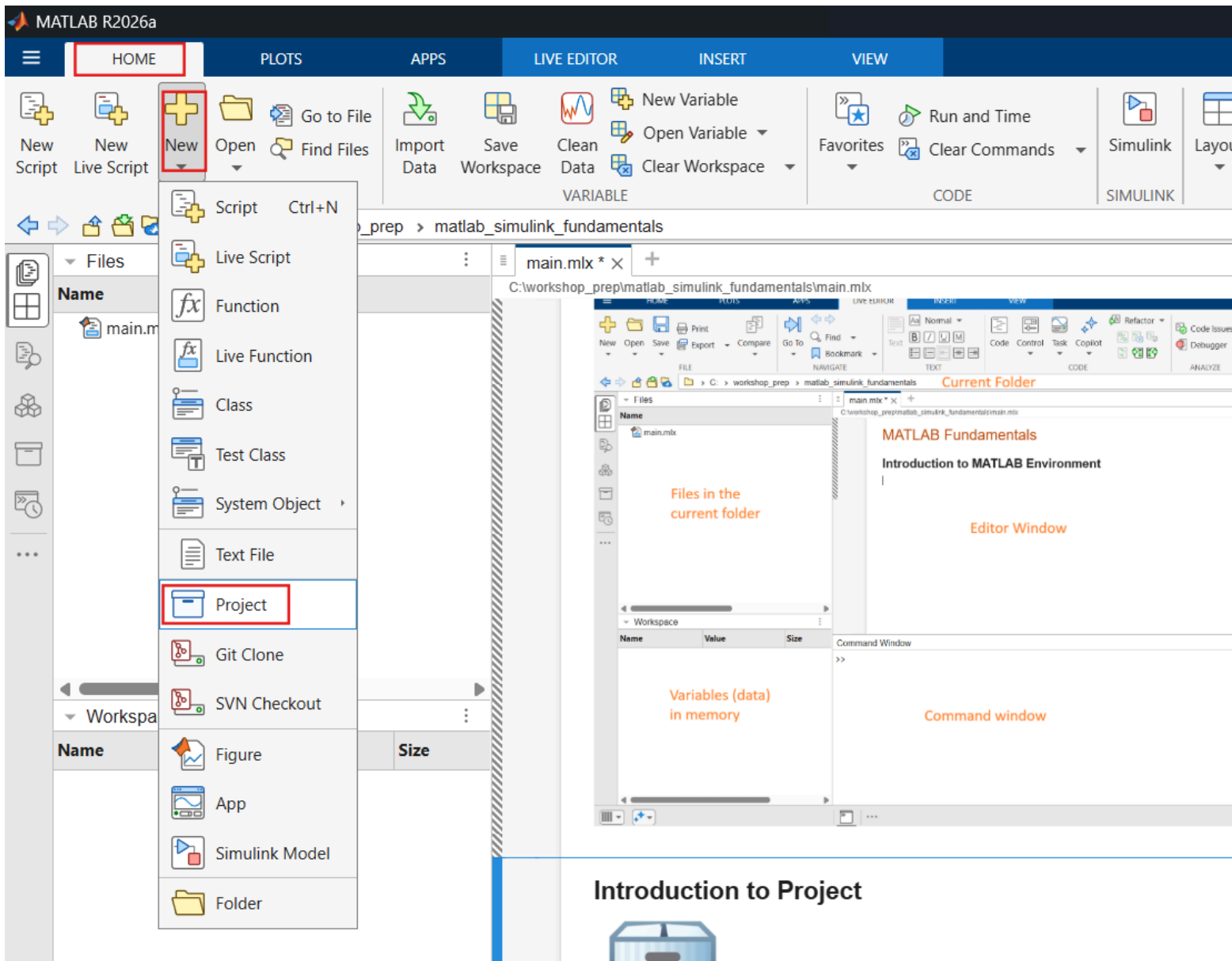
The comment block right below the function line becomes its documentation - try typing: `help addTwoNumbers`

5. Introduction to Project

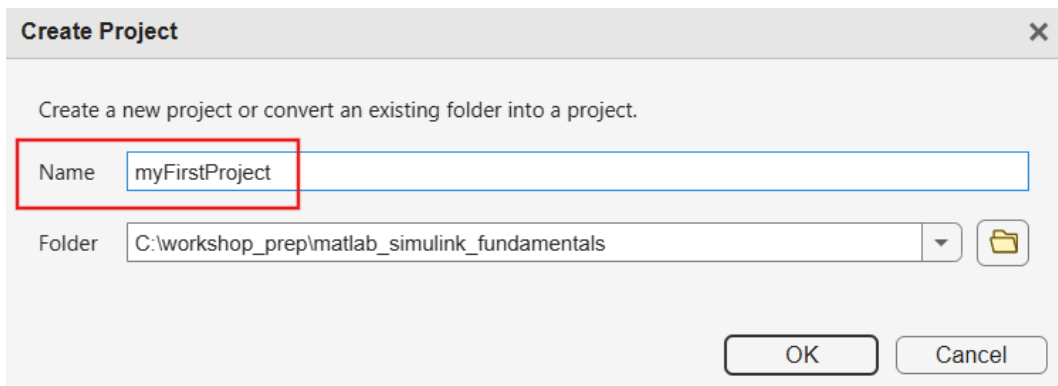
Purpose:

1. Easy Environment Setup - Path, Startup, Shutdown, Shortcuts
2. File Sharing and Access - Support for Source Control, References
3. Tools for Effective Teamwork - Labels, Diff & Merge, Dependency Analysis
4. Project Maturity & Maintainability - Dashboards, Checks and Automated Tasks

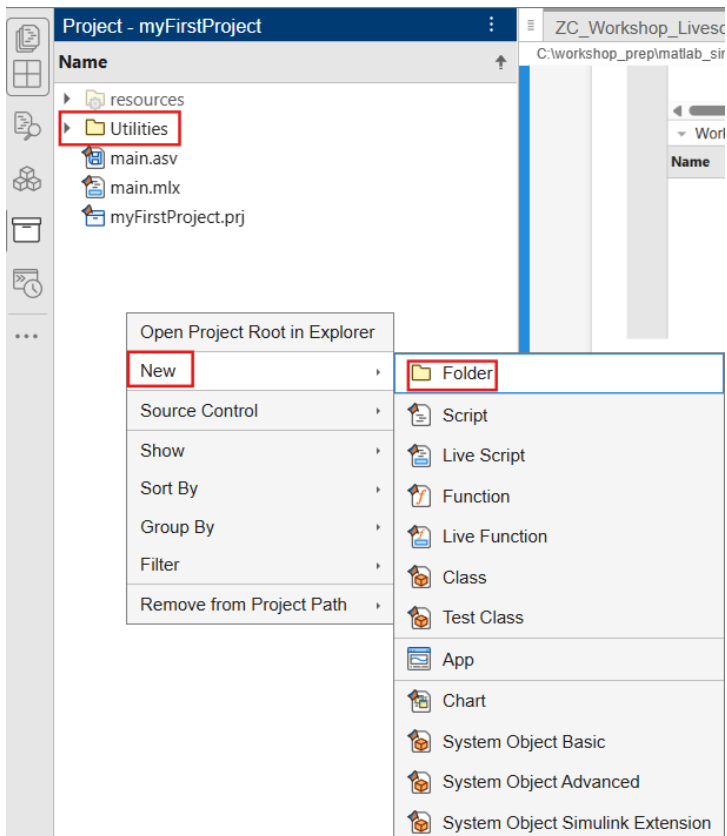
1. Create a new project



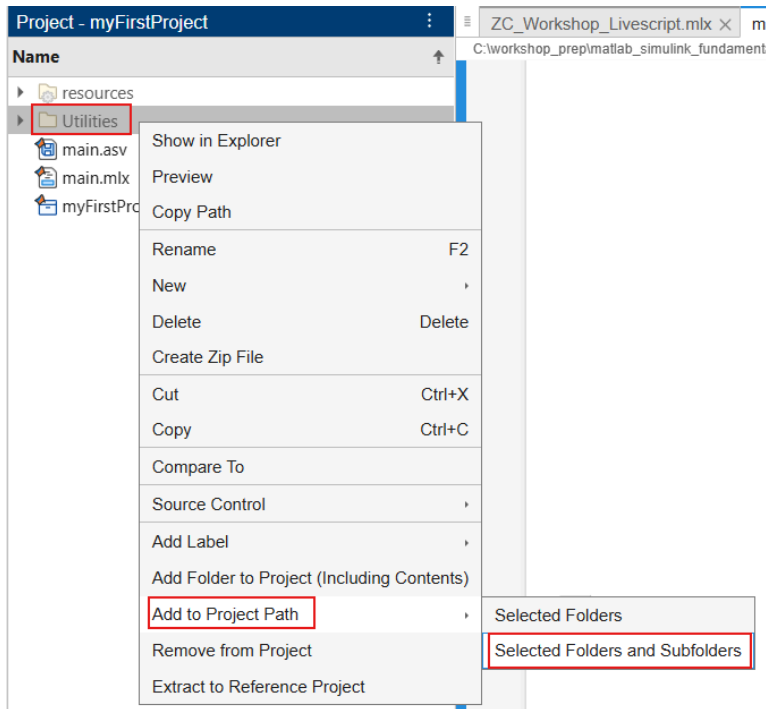
2. Input the project name as "myFirstProject". Save it to any folder that you can access easily.



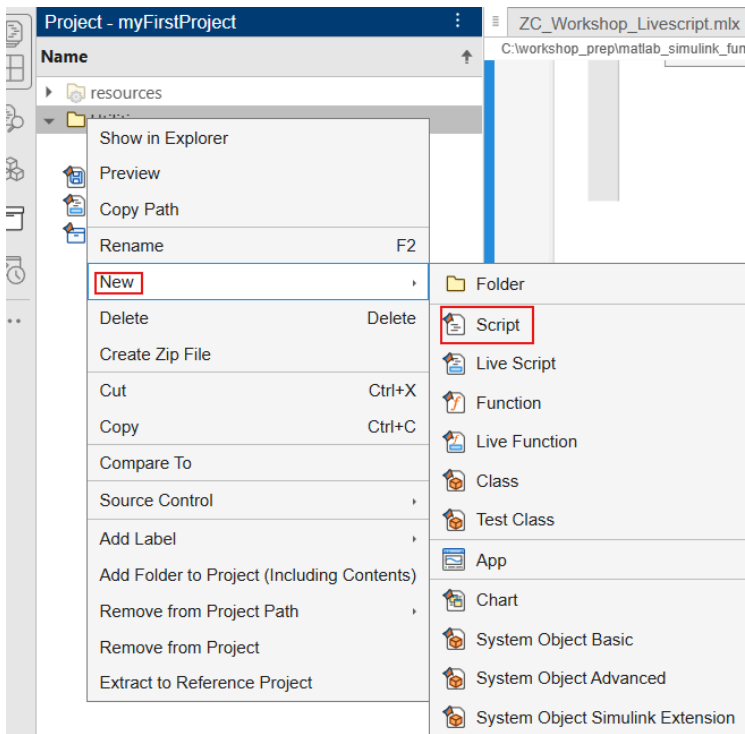
3. Create a folder called Utilities.



4. Add folder to path so that MATLAB can locate the directory. You will be able to see the folder is not grayed out once its in your path.



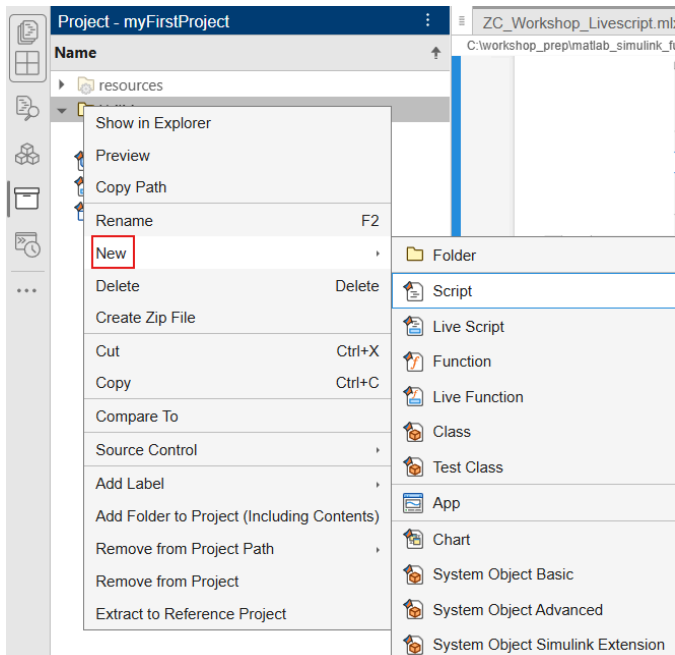
5. Create a new matlab script and name it testStartup.m



6. Inside testStartup.m, put in commands below:

```
disp("This is a startup file.");
open("main.mlx");
disp("You are all set!")
```

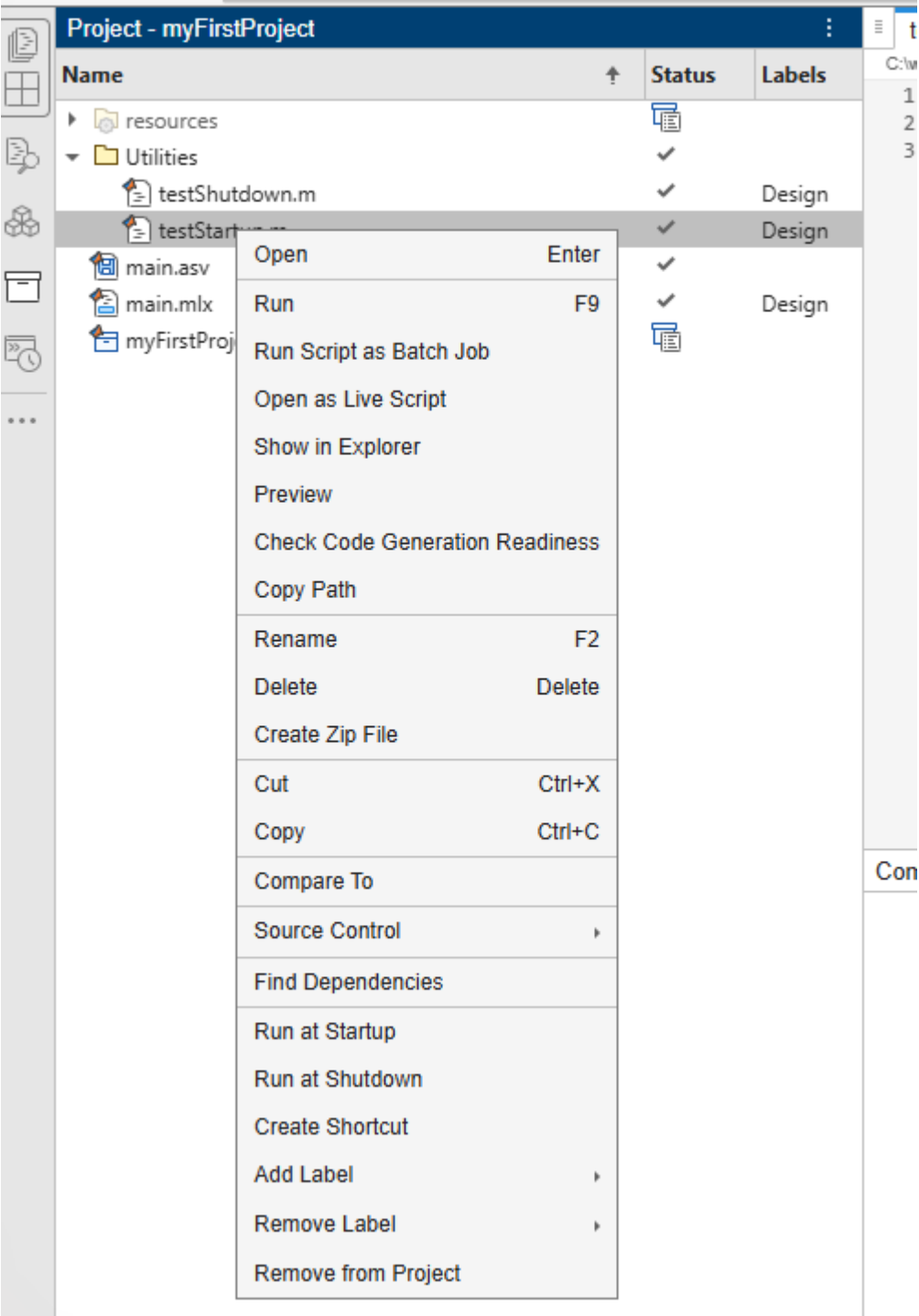
7. Create another matlab script and name it testShutdown.m



Inside testShutdown.m, put in:

```
disp("This is a shutdown file.");
disp("Project closed. See you next time!")
```

8. Register the scripts with the project: right-click testStartup.m in the Project panel and select Run at Startup. MATLAB will now run this script automatically every time the project opens.



9. Repeat for testShutdown.m, this time selecting Run at Shutdown. Small status badges appear next to both files in the Project panel:

Project - myFirstProject			
Name	↑	Status	Labels
resources			
Utilities		✓	
testShutdown.m		✓	Design
testStartup.m		✓	Design
main.asv		✓	
main.mlx		✓	Design
myFirstProject.prj			

10. Test it! Close the project with the Close Project button (or close(currentProject) in the Command Window). The Project panel then lists recent projects for quick access:

Project

No project opened

New Project

Open Project

Create or open a project to manage and share files and settings, find required files, and interact with source control.

[Learn more about projects](#)

Recent Projects

myFirstProject (C:\workshop_prep\matlab_simulink_fundamentals)

ai-mbd-workshop (C:\demo\AI-with-MBD)

DataCenterDesign (C:\demo\simulink-Data-Center-Simulink-25.2.1.1)

ZC_Workshop ...Downloads\system-composer-aero-workshop-master)

BatteryManagementSystem (C:\demo\BatteryManagementSystem)

ai-mbd-workshop ...te Ltd\MATLAB Day\2026\Workshop\AI-with-MBD)

Reference Application Template ...Converter-Controller-HIL-R2025a_v1)

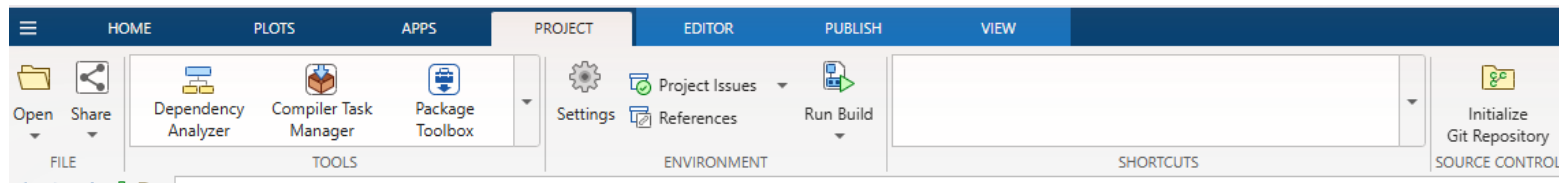
hdl_coder_optimization (C:\demo\hdl_coder_optimization)

Reopen the project - the startup script runs, prints its messages, and opens main.mlx automatically:

```
>> close(currentProject)
>> openProject("C:\workshop_prep\matlab_simulink_fundamentals\myFirstProject.prj");
This is a startup file.
You are all set!
>>
```

9

With the project open, the PROJECT tab offers tools for dependency analysis, packaging, source control integration and build automation:



Under the project tab, click settings to explore further options for the project environment:

